ALA-TOO INTERNATIONAL UNIVERSITY

ALGORITHMIZATION AND PROGRAMMING

PART II

Spring Semester 2025-2026

***

The art of programming is the art of organizing complexity.

— Edsger W. Dijkstra

WORLD THREE

FAST AND STRATEGIC BUT APPROXIMATE

World One → Exact but slow

World Two → Smarter but still exact

World Three introduces a new trade-off:

We accept solutions that are not guaranteed optimal in exchange for dramatic gains in speed and scalability.

This is not lowering standards, this is engineering reality.

GREEDY ALGORITHMS

INTRODUCTION

[7 , 5 , -1 , 1 , -3 , 0]

Find m number that is equal to a largest sum

m = 4

GREEDY ALGORITHMS

SOLUTION

[7 , 5 , -1 , 1 , -3 , 0]

We need to see which 4 number will give largest sum

7 , 5 , 1 , 0 - are selected numbers

7 + 5 + 1 + 0 = 13

GREEDY ALGORITHMS

USE WHEN

1 - GREEDY CHOICE PROPERTY

A global (overall) optimal solution can be reached by choosing the optimal choice at each step

GREEDY ALGORITHMS

USE WHEN

2 - OPTIMAL SUBSTRUCTURE PROPERTY

A problem has an optimal substructure if an optimal solution to the entire problem contains the optimal solutions to the sub-problems

(Everytime system make a choice it is a sub-problem)

GREEDY ALGORITHMS

KNAPSACK PROBLEM

Capacity = 25

Item	Capacity	Value
0	22	19
1	10	9
2	9	9
3	7	6

MAX Value - ?

GREEDY ALGORITHMS

EXAMPLE (KNAPSACK PROBLEM)

Capacity = 25

Item	Capacity	Value
0	22	19
1	10	9
2	9	9
3	7	6

MAX Value - ?

You can select a fractional amount of an item (%)

GREEDY ALGORITHMS

EXAMPLE (KNAPSACK PROBLEM)

Focusing on a smallest capacity approach

First step is simple selection of min possible to fit max value

Item 3 → 7 capacity → 6 = 6 value

Item 2 → 9 Capacity → 9 = 15 Value (16 Capacity)

Item 1 (90%) → 10 Capacity → 8.1 Value = 23.1 value (25 capacity)

GREEDY ALGORITHMS

EXAMPLE (KNAPSACK PROBLEM)

Focusing on a largest value approach

First step is simple selection of min possible to fit max value

Item 0 → 22 capacity → 19 = 19 value

Item 1 (30%) → 3 capacity → 2.7 = 21.7 Value

GREEDY ALGORITHMS

EXAMPLE (KNAPSACK PROBLEM)

THINK ABOUT YOUR CRITERIA

GREEDY ALGORITHMS

EXAMPLE (KNAPSACK PROBLEM)

Capacity = 25

Item	Capacity	Value	Value/Capacity
0	22	19	0.8636
1	10	9	0.9
2	9	9	1
3	7	6	0.857

GREEDY ALGORITHMS

EXAMPLE (KNAPSACK PROBLEM)

Focusing on a ratio approach

Item 2 → 9 Value → 9 Capacity = 9

Item 1 → 9 Value → 10 Capacity = 18

Item 0 (27%) → 5.18 Value = 23.18

RESULT

IDEA

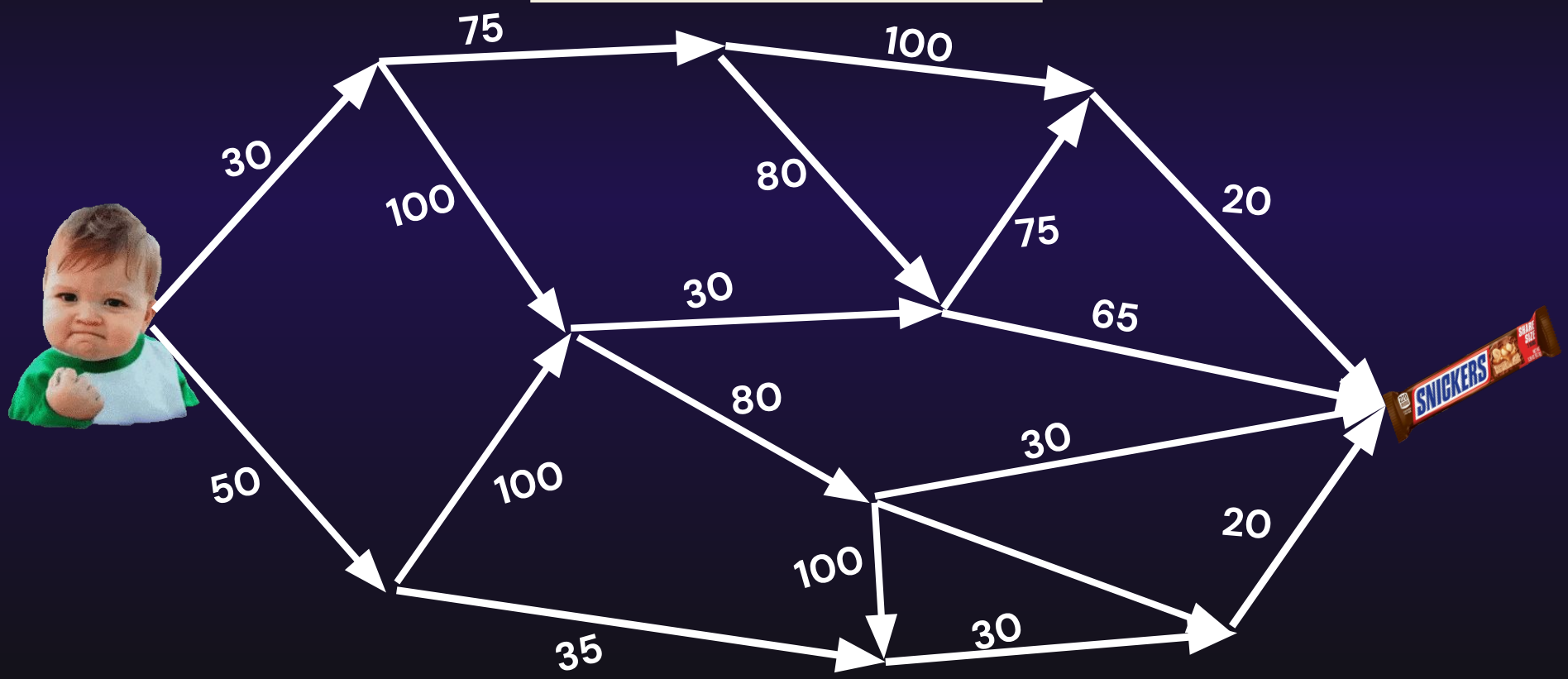
Focusing on a largest value approach - 23.1

Focusing on a smallest capacity approach - 21.7

Focusing on a ratio approach - 23.18

THE PATH

TASK



WHY 3 WORLD ALGORITHMS FAIL AT SCALE

FACTORIAL LIMITS

Many real problems grow factorially or exponentially

Example: Traveling Salesman Problem

Number of routes for n cities:

$$(n - 1)! / 2$$

For 15 cities: ≈ 43 billion routes

For 20 cities: ≈ 60 trillion routes

For 50 cities: ≈ 3 × 10^64 routes

Exact computation becomes physically impossible

World Three exists because scale destroys perfection

COMPLEXITY BARRIER

NP-HARD

A problem is NP-hard if there is no known polynomial-time algorithm and solving it efficiently would solve all NP problems efficiently

Many optimization problems are NP-hard:

- Traveling Salesman
- Knapsack
- Scheduling
- Graph coloring

THE CORE TRADE-OFF

TIME AND QUALITY

We trade optimality for feasibility

Three models:

Exact algorithm → optimal, slow

Heuristic → fast but no guarantee

Approximation → fast but provable bound

World three operates in the last two categories

HEURISTIC ALGORITHMS

GUIDANCE

A heuristic algorithm uses problem-specific knowledge to guide search toward good solutions without guaranteeing optimality

Heuristics are practical decision rules

They reduce search space by making educated guesses

HUMAN ANALOGY

REALITY

Herbert Simon introduced: bounded rationality

Humans do not optimize perfectly. They satisfice they look for a solution that is good enough

World three algorithms mirror this principle

GREEDY ALGORITHMS

IRREVERSIBLE

Greedy strategy: at each step, choose the locally best option.

Never reconsider previous decisions

This makes them:

- Extremely fast
- Memory efficient
- Simple to implement

But potentially wrong globally

WHEN GREEDY IS PROVABLY CORRECT

PROOF

Greedy works when two properties hold:

1 - Greedy choice property

2 - Optimal substructure

Example:

Minimum Spanning Tree (Kruskal, Prim)

Choosing the smallest edge never prevents optimality

WHEN GREEDY FAILS

KNAPSACK EXAMPLE

Capacity = 10

Items:

Value 60, Weight 10

Value 100, Weight 20

Value 120, Weight 30

Greedy by value/weight ratio fails to find optimal combination in some cases

Greedy can be short-sighted.

APPROXIMATION ALGORITHMS

BOUND

Approximation algorithm:

Returns solution within factor α of optimal

If $\alpha = 2$:

- Solution $\leq 2 \times$ optimal
- This gives theoretical confidence

Example:

- 2-approximation for Vertex Cover

WHY APPROXIMATION MATTERS

SCALE

In industry, we need:

- Fast answers
- Predictable performance
- Controlled risk

Approximation algorithms provide formal performance guarantees

Heuristics provide speed without guarantees

DEEP WORKED EXAMPLE

GREEDY

We now examine a concrete numeric example

Cities: A, B, C, D

Distance matrix:

A - B 10

A - C 15

A - D 20

B - C 35

B - D 25

C - D 30

GREEDY TSP STEP-BY-STEP

RESULT

Start at A:

Closest city: B (10)

From B → closest unvisited: D (25)

From D → C (30)

Return C → A (15)

Total = 10 + 25 + 30 + 15 = 80

EXACT SOLUTION COMPARISON

ENUMERATION

Check all routes:

Optimal route = 80

In this case greedy worked.

But change one edge slightly:

If C-A = 5 instead of 15

Greedy route changes and may no longer be optimal.

Small structural change can break greedy

LESSON FROM EXAMPLE

STRUCTURE

Greedy depends on structure

When structure aligns → optimal
When structure shifts → suboptimal

World three requires understanding problem properties

INDUSTRY EXAMPLE

ROUTING

Internet routing does not compute global optimal paths.

Protocols use heuristics:

- Shortest path estimates
- Load balancing approximations
- Local updates

Global recomputation would be too slow

LARGE-SCALE SYSTEMS

BILLIONS STREAMING ONLINE

Streaming platforms:

Billions of events per day.

Optimization must be:

Incremental
Heuristic
Fast

Exact recomputation is infeasible.

RISK AND RESPONSIBILITY

ERROR IMPACT ETHICS

Approximate decisions can affect:

- Financial systems
- Medical scheduling
- Resource allocation

Speed must be balanced with responsibility.

World Three is powerful but must be applied carefully.

WHAT IF

10,000 CITIES 2 SECONDS DEADLINE

You must compute delivery route for 10,000 cities in 2 seconds

Exact algorithm: impossible
Backtracking: impossible

Greedy heuristic: feasible

98% optimal in 0.5 seconds vs 100% optimal in 3 years

Which do you choose?

This is engineering

FORMAL SUMMARY

HEURISTIC GREEDY APPROXIMATE

World three includes:

Heuristics → no guarantee

Greedy → local decisions

Approximation → provable bound

All trade optimality for scalability

HIERARCHY OF WORLDS

EXACT PRUNED APPROXIMATE

World One → brute force

World Two → constraint-based exact

World Three → scalable approximation

Each world solves different scale problems

THE PROFESSIONAL PERSPECTIVE

ENGINEERING TRADE-OFF MATURITY

A mature engineer asks:

Not “Is it perfect?”
But “Is it appropriate for the scale?”

World Three is algorithmic maturity.

T007

REQUIREMENTS

Generate random cities (n = 10–12)

Compute route using greedy

Compute exact solution for n ≤ 8

Compare distance

Compare time

T007

ANALYSIS REPORT

Report:

Distance difference (%)

Execution time difference

Observed failure cases

Reflection on trade-off

EXTENSION OPTION

T007

Improve greedy by running multiple starting cities

Choose best result

Observe improvement.

WHEN WORLD TWO IS NOT NEEDED

RIGHT TOOL, RIGHT PROBLEM

DISCUSSION

WHEN TO USE

Use when:

- Scale is large
- Time is limited
- Exactness unnecessary

Avoid when:

- Safety-critical systems
- Small input sizes
- Exact result required

TAKEAWAY

INTELLIGENCE

World three is not careless it is controlled imperfection designed for real-world scale

NEXT WEEK PREVIEW

COMING SOON

Metaheuristics

Genetic algorithms

Simulated annealing

AI optimization systems